

Garbage Collection in Macaulay2

One collector for everything

Macaulay2 uses the **Boehm–Demers–Weiser conservative collector** (bdwgc, “libgc”) as its *only* memory manager for language-level and engine-level objects. There is no reference counting and almost no manual freeing anywhere in the system.

- **Configuration** lives in one header, `include/M2/gc-include.h`. It defines `GC_THREADS` before including `<gc/gc.h>`, which remaps `pthread_create` to `GC_pthread_create` — so every thread registers itself with the collector automatically. The initial heap is 70 MB.
- **Startup:** `bin/main.cpp` calls `GC_INIT()` and grows the heap to at least 150 MB + 8 MB per allowable thread.
- **Allocation wrappers:** everything funnels through `e/interface/m2-mem.cpp`. `getmem` calls `GC_MALLOC` (memory the collector scans for pointers); `getmem_atomic` calls `GC_MALLOC_ATOMIC` (pointer-free memory the collector skips, used for strings and number data).
- **Freeing is a no-op:** `freemem` does nothing — reclamation is left entirely to the collector. From the M2 language, `collectGarbage` forces a collection and `GCstats` reports heap statistics.

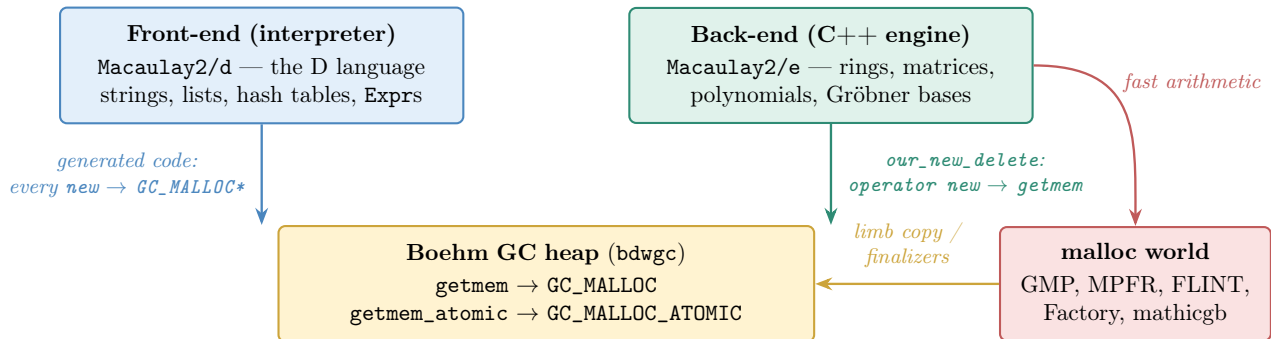


Figure 1: The two halves of Macaulay2 allocate from the same conservative GC heap. Only the engine talks to malloc-backed arithmetic libraries, and that memory is bridged back into the GC world (Figure 2).

Front-end: the interpreter (Macaulay2/d)

The interpreter is written in Macaulay2’s own *D language*, and its translator (SCC, `c/cprint.c`) **emits a GC call for every new expression**. Types containing pointers — expressions, sequences, hash tables — compile to `GC_MALLOC`; pointer-free data such as strings and integer arrays compile to `GC_MALLOC_ATOMIC`. As a result, *every* value the interpreter creates lives in the GC heap: there is no separate front-end allocator and no manual freeing at all.

- **Finalizers guard foreign resources only.** `GC_REGISTER_FINALIZER` is attached exclusively to objects owning non-GC state: task cells (so an unreachable running task gets canceled, `d/pthread.d`), mutexes and RW-locks (`d/pthread0.d`), FFI closures and foreign GMP/MPFR values (`d/ffi.d`), and MySQL handles (`d/mysql.d`). The user-facing hook is `registerFinalizer` (`d/actors2.dd`).
- **Threads are GC-transparent.** The thread supervisor (`system/supervisor.cpp`) spawns task threads with plain `pthread_create`; the `GC_THREADS` remap registers them automatically. Supervisor containers are `gc_allocator`-backed STL maps and sets (`system/gc_std.hpp`), so the collector

scans them like everything else.

Back-end: the C++ engine (Macaulay2/e)

Engine classes opt into the GC heap by inheriting `our_new_delete` (`e/newdelete.hpp`), which overloads `operator new` to call `getmem`. Its subclass `our_gc_cleanup` also registers a finalizer, so C++ destructors run when the collector reclaims the object (used by `MutableEngineObject`). GC-aware containers come from `gc_vector<T>`, a `std::vector` with `gc_allocator`. Polynomial terms (`Nterm`, `rings/poly.cpp`) are GC-allocated; monomial exponent vectors use *atomic* GC memory since they contain no pointers. The legacy `stash/slab` free-list allocators (`e/mem.hpp`) are short-circuited to plain GC calls.

The interesting part is that the engine **straddles two memory regimes**:

- **Deliberate GC bypasses.** GMP, MPFR, FLINT, Factory, mathicgb, and memtailor all allocate with their own `malloc` — M2 never calls `mp_set_memory_functions`. Scratch arenas (`MemoryBlock.hpp`, wrapping memtailor’s `memt::Arena`) back hot paths like F4 without touching the collector.
- **Crossing the boundary.** When a multiprecision result becomes a long-lived `ring_elem`, its limbs are *copied into GC memory* by `mpz_reallocate_limbs` / `mpfr_reallocate_limbs` and the `moveTo_gmp*` helpers (`e/interface/gmp-util.h`).
- **The ARing ownership rule** (`e/basic-rings/arings.md`): working `ElementType` temporaries live *outside* the GC and are freed via `clear`, but any pointer packed into a `ring_elem` handed to the engine *must* reference GC-owned storage. For example, `aring-ZZ-flint.hpp` computes with `malloc`-backed `fmpz` and converts through a GC-allocated `mpz` in `to_ring_elem`.
- **Finalizers reconcile the two worlds.** `e/finalize.cpp` registers finalizers on interned Gröbner bases, resolutions, monomial ideals, and mutable matrices. When the collector frees the GC-tracked wrapper, the finalizer `deletes` it, releasing the `malloc`-backed FLINT/Factory/mathicgb resources it owns.

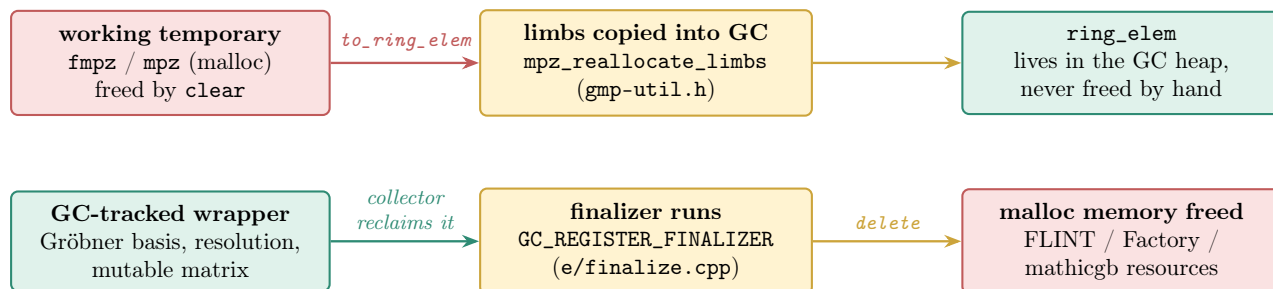


Figure 2: Bridging the GC and malloc worlds in the engine. *Top:* multiprecision values enter the GC heap by copying their limbs. *Bottom:* malloc-backed library resources are released by finalizers when their GC-tracked owner becomes unreachable.

Summary

The **front-end trusts the collector completely**: every interpreter value is GC-allocated by generated code, with finalizers only guarding foreign handles. The **back-end straddles two regimes**: engine objects live in the GC heap, while fast arithmetic runs on malloc-backed libraries whose memory is bridged in by limb-copying helpers and reclaimed through finalizers.